Lab7.1: Working with Cron Jobs

Verify Cron Service

1. Check if cron is running:

```
systemctl status cron
```

2. If not running, start it:

```
sudo systemctl start cron
sudo systemctl enable cron
```

Create a Simple Script

1. Create a script called `hello_cron.sh`:

```
vi ~/hello_cron.sh
```

You Can use any editor you like

2. Add the following content:

```
#!/bin/bash
echo "Cron ran at $(date)" >> ~/cron_output.log
```

3. Make it executable:

```
chmod +x ~/hello_cron.sh
```

Question: Why is it necessary to make the script executable?

Schedule the Script with Cron

The first time you run this you will have a choice of editors.

I have not used nano but it seams the easiest

1. Open your crontab:

```
crontab -e
```

2. Add this line to run the script every minute:

```
* * * * * /home/yourusername/hello_cron.sh
```

Whereever you saved your script

3. Save and exit.

Question: What does each asterisk represent in the cron expression?

Verify Execution

1. Wait 2-3 minutes (this can seam like a long time 😊).
2. Check the output log:

```
cat ~/cron_output.log
```

> **Question: How many entries do you see, and what does this tell you about the job frequency?**

Modify the Schedule

1. Edit your crontab again:

```
crontab -e
```

2. Change the schedule to run every 5 minutes:

```
*/5 * * * * /home/yourusername/hello_cron.sh
```

> **Question: Write the cron expression for "every 10 minutes."**

Use a Special String

1. Modify the crontab to run the script once daily at midnight using a special string:

```
@daily /home/yourusername/hello_cron.sh
```

> **Question: What is the equivalent standard cron syntax for @daily?**

Troubleshooting Exercise

Intentionally break the script by removing the shebang line (`#!/bin/bash`)

Run the cron job again and check if it executes correctly

Fix the issue and verify the

> **Question: What error did you encounter, and how did you resolve it?**

Try looking at

/var/log/sys/log

```
grep CRON /var/log/syslog
```

Clean Up

1. Remove the cron job:

```
crontab -e
```

(Delete the line referencing `hello_cron.sh`)

2. Confirm removal:

```
crontab -l
```

Questions

1. What are the risks of scheduling cron jobs without proper logging?
2. How would you prevent two long-running cron jobs from overlapping?
3. Why might `@reboot` be useful in a production environmenls t?

You may want to look at lock files